

HOMEWORK 1

Implementing the Semantics of IMP

CS4211 · Formal Methods for Software Engineering

Due: **September 8, 2026** · 100 marks, plus 10 bonus

In this homework, you are asked to implement the big-step and small-step semantics of IMP. This handout contains the complete set of semantic rules used by the assignment, including the extensions for division and arrays. Treat the section “Definitive semantic rules” below as the ground truth for this homework. Your task is to turn each rule into a corresponding case in an interpreter.

You may use **Python, Java, or C++**. We have supplied a starting point for each language. The starting point reads and writes JSON, represents the abstract syntax trees, and dispatches each request to the appropriate function. You are not asked to write a lexer, a parser, or a JSON library. You are asked to implement the semantics.

The homework is arranged so that you can work through it one part at a time. Please begin with the setup below, even if you intend to use your own testing scripts later.

Download and inspect the starting point

Download `cs4211-hw1.zip` from the Homework 1 page on Canvas. Unpack it and enter the resulting directory:

```
unzip cs4211-hw1.zip
cd cs4211-hw1
```

The first command extracts the archive. The second changes the current directory to the extracted homework directory. Run all remaining commands in this handout from that directory unless stated otherwise.

You should see the following files and directories:

<code>handout/HW1.pdf</code>	this handout
<code>README.md</code>	concise setup and submission guide
<code>SPEC.md</code>	exact JSON and AST formats
<code>adapters/python/</code>	Python starting point
<code>adapters/java/</code>	Java starting point
<code>adapters/cpp/</code>	C++ starting point
<code>tests/public/</code>	public tests and expected outputs
<code>tests/malformed/</code>	examples rejected before your program runs
<code>validate.py</code>	runs your program on the tests
<code>render.py</code>	renders derivations and executions
<code>AI-USE.md</code>	disclosure template

The handout explains what you need to implement and in what order. `SPEC.md` is the reference for the exact request and response formats. You will use both: read the handout first, and consult the relevant part of `SPEC.md` whenever you need the precise shape of a JSON object.

The files in `tests/public/` are called *public tests* because they are included in the student

archive: you can open each test and inspect both its request and its expected response. The tests used for marking also include *hidden tests*, which are not given to you. The hidden tests use the same semantics and JSON formats, but apply them to different programs.

The program `validate.py` is the supplied test driver. For every selected test, it starts your implementation, sends the test's request to the implementation on standard input, reads one JSON response from standard output, and compares that response with the test's expected response. It does not inspect your source code. Thus, the same driver can test a Python, Java, or C++ implementation.

Each language directory is itself divided into files. The division is the same in all three languages:

Responsibility	Files
Typed IMP objects	<code>model.py</code> , <code>Model.java</code> , or <code>model.hpp/model.cpp</code>
JSON conversion	<code>codec.py</code> , <code>Codec.java</code> , or <code>codec.hpp/codec.cpp</code>
Arithmetic helpers	<code>arithmetic.py</code> , <code>Arithmetic.java</code> , or <code>arithmetic.hpp</code>
C++ integer type	<code>integer.hpp</code> (supplied; no external library needed)
Your semantic rules	the big-step, small-step, and analysis files
Process entry point	<code>solution.py</code> , <code>Solution.java</code> , or <code>solution.cpp</code>

The typed-object, codec, arithmetic, and entry-point files are supplied. They are separated so that, while implementing a semantic rule, you work with an object such as `Assign` or `Num`, not with a large untyped JSON dictionary.

Choose a language and make your working copy

Choose one language and copy its entire adapter directory to a new directory named `submission`. The adapters contain several files that work together, so do not copy only `solution.py`, `Solution.java`, or `solution.cpp`. Run exactly one of the following commands from the top-level `cs4211-hw1` directory:

```
# Choose exactly one language.
cp -R adapters/python submission
# cp -R adapters/java submission
# cp -R adapters/cpp submission
```

The command `cp -R` copies the selected directory, including all of its files. The original files remain under `adapters/`; use the copy under `submission/` for your implementation. Do this once, before you begin editing. If you use Windows without a `cp` command, make the same copy in File Explorer and rename the copied directory `submission`.

For example, after selecting Python, `submission/` contains `solution.py`, `model.py`, `codec.py`, `big_step.py`, `small_step.py`, `analysis.py`, and the arithmetic helper. Java and C++ have the corresponding files listed in the responsibility table above. Do not combine files from two language adapters.

Build and run the supplied program

Build the program, if necessary, and ask `validate.py` to run the tests in `tests/public/`. Use the commands for the language you copied.

```
# Python
python3 validate.py \
  --cmd "python3 submission/solution.py" \
  tests/public

# Java
mkdir -p build/java
javac submission/*.java -d build/java
python3 validate.py \
  --cmd "java -cp build/java Solution" \
  tests/public

# C++
mkdir -p build
g++ -std=c++17 -O2 submission/*.cpp -o build/solution
python3 validate.py \
  --cmd "./build/solution" \
  tests/public
```

Here is what these commands do. Python runs the supplied entry-point file directly; that file imports the other Python files from the same directory. For Java, `mkdir` creates a directory for compiled class files, `javac submission/*.java` compiles every Java source file in your working copy, and `-d build/java` places the resulting classes in that directory. The command `java -cp build/java Solution` then starts the compiled `Solution` class. For C++, `-std=c++17` selects C++17, `-O2` enables optimization, `submission/*.cpp` compiles every C++ implementation file in your working copy, and `-o build/solution` names the resulting executable.

The final line in each case runs the test driver. The value following `-cmd` is the shell command that the driver should use to start your implementation. The final argument, `tests/public`, tells it which test directory to read. The driver runs every graded test in that directory unless you give it a further option such as `-only` or `-part`.

Apart from a compiler or interpreter for your chosen language, you only need Python 3.8 or later to run `validate.py` and `render.py`. No third-party package is required. The displayed commands use a POSIX shell. On Windows, run them in WSL or Git Bash, or translate the line continuations and `copy/build` commands to PowerShell; the Python tools themselves are cross-platform.

What should happen before you implement anything?

Most semantic functions in the starting point deliberately raise a “not implemented” exception. If you run `validate.py` on every test in `tests/public/`, the summary should initially be:

graded: 1/64 passed

by part:

A	0/15
B	1/21
C	0/ 6
D	0/18
bonus	0/ 4

The line graded: 1/64 passed says that one of the 64 included graded tests passed. The table then separates the same result by marking part. The one passing test is b01. It asks what happens when the command is already skip. The supplied code that receives each JSON request knows that skip is final, so it can answer this request without calling any function marked TODO. This first result is useful: it confirms that the program starts, reads a request, and writes a response in the required format. If your first run does not pass b01, fix the setup before changing the semantics.

Understand one test before writing code

Open tests/public/a01.json. Each test contains:

- an id and a short description;
- a part, which determines where its marks belong;
- a request, which is sent to your program; and
- an expected response.

Test a01 asks for the big-step execution of $r := 7$ from the empty state. Its program field contains the following abstract syntax tree, written as JSON:

```
{
  "k": "assign",
  "x": "r",
  "e": {
    "k": "num",
    "n": 7
  }
}
```

The braces delimit JSON objects, each colon separates a field name from its value, and the nested braces under e describe a child node of the tree. Every node of a syntax tree is represented by one such object. The field k is short for *kind*: it says which syntactic form the node represents. The outer node has "k": "assign", so it represents an assignment command. Its field x contains the name of the variable being assigned, namely r. Its field e contains the expression on the right-hand side.

That expression is itself a syntax-tree node and therefore has its own k field. Here its kind is num, meaning a numeral, and its n field gives the numeral's value, 7. Thus, read from the inside out, the inner object represents the expression 7, and the outer object represents the command $r := 7$. The two occurrences of k belong to two different nodes; they are not two names for the same field. Section 1 of SPEC.md lists the corresponding fields for every other syntactic form.

The JSON format is the external representation used by the tests. You do not inspect k , x , or e inside the semantic functions. The supplied codec first converts this JSON tree into typed objects:

Language	Object received by the command evaluator
Python	<code>Assign("r", Num(7))</code>
Java	<code>new Model.Assign("r", new Model.Num(7))</code>
C++	<code>imp::Assign</code> whose expression is an <code>imp::Num</code>

If you choose Python and open `model.py`, you will see `@dataclass(frozen=True)` above many of these class definitions. The `@` introduces a Python *decorator*: `dataclass` asks Python to generate routine methods such as the constructor and equality comparison. The option `frozen=True` prevents you from assigning new values to an object's fields after it has been created. Thus `Num(7)` constructs a small, immutable AST node whose value field is 7. You use these classes as supplied; you do not need to change the decorator.

The codec is the only place that reads the short JSON field names. In the `Assign` case, your command evaluator recursively evaluates the typed expression field and updates the typed variable field. The program is already parsed; you never recover `r := 7` from a string and you do not write JSON-handling code.

Your implementation should return the final state in which r has value 7, together with a derivation using `[Num]` and `[Asgn]`.

The supplied program `render.py` is a display tool. It does not run or test your implementation. With `-from-test`, it reads the expected response stored in the named test file and prints that response in a readable form: a proof tree for a derivation, or a table for an execution. For example, the following command displays the expected proof tree for `a01`:

```
python3 render.py --from-test tests/public/a01.json
```

To run your implementation on only this test, use the line for your chosen language:

```
# Python
python3 validate.py --cmd "python3 submission/solution.py" \
  --only a01 tests/public

# Java
python3 validate.py --cmd "java -cp build/java Solution" \
  --only a01 tests/public

# C++
python3 validate.py --cmd "./build/solution" \
  --only a01 tests/public
```

The quoted value after `-cmd` is exactly the command that starts your program. The option `-only a01` filters the test directory to the test whose id is `a01`. Add `-verbose` when you want the driver to print the complete difference between your JSON response and the expected response, or the complete error output when your program exits unsuccessfully. For Python, you can check the syntax of every submission file without running a test:

```
python3 -m py_compile submission/*.py
```

This reports the filename and line number of the first syntax error.

For brevity, the remaining examples use the Python command. If you chose Java or C++, replace only "python3 submission/solution.py" by the corresponding quoted value above; keep the other arguments unchanged.

Which files should you change? The files named below contain the semantic functions marked TODO:

	Part A	Part B	Part C/bonus
Python	big_step.py	small_step.py	analysis.py
Java	BigStep.java	SmallStep.java	Analysis.java
C++	big_step.cpp	small_step.cpp	analysis.cpp

You should normally change only these three files inside submission/. The other files define the typed AST/state objects, translate JSON, provide arithmetic helpers, and run the process interface. Read them to understand the available fields and helper methods, but do not duplicate their work in your semantic functions. Keep adapters/ unchanged; it is your reference copy of the starting point.

What does the validator compare? JSON key order and whitespace do not matter. In a derivation node, the validator compares rule, in, prem, val, and out. The optional subj field is used by render.py, but it is not marked. For a stuck response, only the status is marked; you may write the explanation in your own words. Section 6 of SPEC.md gives the complete comparison rule.

Marks and a suggested order of work

Part		Marks
A	Big-step semantics and derivations	25
B	Small-step semantics and traces	25
C	Detecting repeated configurations	10
D	Division, wrapping arithmetic, and arrays	20
	Written questions	20
Total		100
Bonus	Non-deterministic choice	up to 10

Marks are assigned by *part*. The option `-part A`, for example, selects every test whose `part` field is A; it need not select one particular kind of request. Work in the order A, B, C, D:

```
python3 validate.py --cmd "python3 submission/solution.py" --part A tests/public
python3 validate.py --cmd "python3 submission/solution.py" --part B tests/public
python3 validate.py --cmd "python3 submission/solution.py" --part C tests/public
python3 validate.py --cmd "python3 submission/solution.py" --part D tests/public
```

Each test also has a *group*, such as `step`, `run`, or `classify`. A group describes the kind of request; it is not a marking part. For example, some Part D tests are `step` and `run` requests. This is why `-part B` is the right command while completing Part B. Use `-group step` only when you deliberately want to test that request mode across more than one part.

The letter in a test ID identifies a test series, not necessarily its marking part. The complete public-test mapping is:

Marking part	Public IDs	Main request modes
A	a01–a15	bigstep
B	b01–b14, c01–c07	step, run
C	d01–d06	classify
D	e01–e18	several modes using extensions
bonus	f01–f04	explore

The tests included in the archive are only part of the tests used for marking. Passing them is a useful check of your implementation, but the hidden tests apply the same rules to different programs.

Definitive semantic rules

This section is the complete semantic specification for the assignment. Treat these rules, side conditions, and evaluation orders as definitive. You do not need to consult the lecture notes to decide how a program in this homework behaves. `SPEC.md` remains definitive for the JSON interface.

States are written σ . Variable lookup is total: $\sigma(x) = 0$ when x has no stored non-zero binding. An array name u denotes a fixed-length zero-indexed array; $|\sigma(u)|$ is its length and $\sigma(u)(z)$ is its element at index z . Canonical states omit zero-valued variable bindings but retain every declared array and all of its elements.

The request's arithmetic mode is written m . The partial operation $\text{arith}_m(\circ, n_1, n_2)$ applies $\circ \in \{+, -, *, /\}$ using either mathematical integers or signed 32-bit wrapping, as selected by m . Division truncates toward zero and is undefined when $n_2 = 0$. $\text{cmp}(=, n_1, n_2)$ and $\text{cmp}(\leq, n_1, n_2)$ have their usual Boolean meanings; $\text{con}(\text{and}, t_1, t_2)$ and $\text{con}(\text{or}, t_1, t_2)$ are the usual Boolean operations.

If a rule needs an undefined arithmetic operation or a failed array-bounds side condition, it has no conclusion. In an execution this is *stuck*. The command `skip`, by contrast, is final and is never stuck.

Big-step arithmetic expressions

The judgement $\langle a, \sigma \rangle \Downarrow n$ says that arithmetic expression a has value n in state σ . Expressions do not change the state.

$$\begin{array}{c}
\frac{}{\langle n, \sigma \rangle \Downarrow n} \text{[Num]} \qquad \frac{}{\langle x, \sigma \rangle \Downarrow \sigma(x)} \text{[Var]} \\
\\
\frac{\langle a_1, \sigma \rangle \Downarrow n_1 \quad \langle a_2, \sigma \rangle \Downarrow n_2}{\langle a_1 + a_2, \sigma \rangle \Downarrow \text{arith}_m(+, n_1, n_2)} \text{[Add]} \\
\\
\frac{\langle a_1, \sigma \rangle \Downarrow n_1 \quad \langle a_2, \sigma \rangle \Downarrow n_2}{\langle a_1 - a_2, \sigma \rangle \Downarrow \text{arith}_m(-, n_1, n_2)} \text{[Sub]} \\
\\
\frac{\langle a_1, \sigma \rangle \Downarrow n_1 \quad \langle a_2, \sigma \rangle \Downarrow n_2}{\langle a_1 * a_2, \sigma \rangle \Downarrow \text{arith}_m(*, n_1, n_2)} \text{[Mul]} \\
\\
\frac{\langle a_1, \sigma \rangle \Downarrow n_1 \quad \langle a_2, \sigma \rangle \Downarrow n_2 \quad n_2 \neq 0}{\langle a_1 / a_2, \sigma \rangle \Downarrow \text{arith}_m(/, n_1, n_2)} \text{[Div]} \\
\\
\frac{\langle i, \sigma \rangle \Downarrow z \quad 0 \leq z < |\sigma(u)|}{\langle u[i], \sigma \rangle \Downarrow \sigma(u)(z)} \text{[Arr-Read]}
\end{array}$$

For every binary operator, both premises use the same input state. They are evaluated left to right by the supplied implementation interface, although a successful big-step derivation records both premises rather than intermediate expression configurations.

Big-step Boolean expressions

The judgement $\langle b, \sigma \rangle \Downarrow t$ has $t \in \{\text{true}, \text{false}\}$. Comparisons evaluate their arithmetic operands left to right. Boolean connectives are *strict*: they evaluate both operands, left first, even when the left value alone determines the truth value.

$$\begin{array}{c}
\frac{}{\langle \text{true}, \sigma \rangle \Downarrow \text{true}} [\text{True}] \quad \frac{}{\langle \text{false}, \sigma \rangle \Downarrow \text{false}} [\text{False}] \\
\\
\frac{\langle a_1, \sigma \rangle \Downarrow n_1 \quad \langle a_2, \sigma \rangle \Downarrow n_2}{\langle a_1 = a_2, \sigma \rangle \Downarrow \text{cmp}(=, n_1, n_2)} [\text{Eq}] \\
\\
\frac{\langle a_1, \sigma \rangle \Downarrow n_1 \quad \langle a_2, \sigma \rangle \Downarrow n_2}{\langle a_1 \leq a_2, \sigma \rangle \Downarrow \text{cmp}(\leq, n_1, n_2)} [\text{Leq}] \\
\\
\frac{\langle b, \sigma \rangle \Downarrow t}{\langle \text{not } b, \sigma \rangle \Downarrow \neg t} [\text{Not}] \\
\\
\frac{\langle b_1, \sigma \rangle \Downarrow t_1 \quad \langle b_2, \sigma \rangle \Downarrow t_2}{\langle b_1 \text{ and } b_2, \sigma \rangle \Downarrow \text{con}(\text{and}, t_1, t_2)} [\text{And}] \\
\\
\frac{\langle b_1, \sigma \rangle \Downarrow t_1 \quad \langle b_2, \sigma \rangle \Downarrow t_2}{\langle b_1 \text{ or } b_2, \sigma \rangle \Downarrow \text{con}(\text{or}, t_1, t_2)} [\text{Or}]
\end{array}$$

Big-step commands

The judgement $\langle c, \sigma \rangle \Downarrow \sigma'$ says that command c terminates from σ in σ' . Premises are evaluated in the order shown. The required derivation labels are exactly the names printed beside the rules.

$$\begin{array}{c}
\frac{}{\langle \text{skip}, \sigma \rangle \Downarrow \sigma} [\text{Skip}] \\
\\
\frac{\langle a, \sigma \rangle \Downarrow n}{\langle x := a, \sigma \rangle \Downarrow \sigma[x \mapsto n]} [\text{Asgn}] \\
\\
\frac{\langle c_1, \sigma \rangle \Downarrow \sigma_1 \quad \langle c_2, \sigma_1 \rangle \Downarrow \sigma_2}{\langle c_1; c_2, \sigma \rangle \Downarrow \sigma_2} [\text{Seq}] \\
\\
\frac{\langle b, \sigma \rangle \Downarrow \text{true} \quad \langle c_1, \sigma \rangle \Downarrow \sigma'}{\langle \text{if } b \text{ then } c_1 \text{ else } c_2, \sigma \rangle \Downarrow \sigma'} [\text{If-True}] \\
\\
\frac{\langle b, \sigma \rangle \Downarrow \text{false} \quad \langle c_2, \sigma \rangle \Downarrow \sigma'}{\langle \text{if } b \text{ then } c_1 \text{ else } c_2, \sigma \rangle \Downarrow \sigma'} [\text{If-False}] \\
\\
\frac{\langle b, \sigma \rangle \Downarrow \text{false}}{\langle \text{while } b \text{ do } c, \sigma \rangle \Downarrow \sigma} [\text{While-False}] \\
\\
\frac{\langle b, \sigma \rangle \Downarrow \text{true} \quad \langle c, \sigma \rangle \Downarrow \sigma_1 \quad \langle \text{while } b \text{ do } c, \sigma_1 \rangle \Downarrow \sigma_2}{\langle \text{while } b \text{ do } c, \sigma \rangle \Downarrow \sigma_2} [\text{While-True}] \\
\\
\frac{\langle i, \sigma \rangle \Downarrow z \quad 0 \leq z < |\sigma(u)| \quad \langle a, \sigma \rangle \Downarrow n}{\langle u[i] := a, \sigma \rangle \Downarrow \sigma[u[z] \mapsto n]} [\text{Arr-Write}]
\end{array}$$

In [Arr-Write], the index is evaluated and checked before the value expression is evaluated. Both expressions use the original state because expressions have no side effects.

Small-step arithmetic expressions

The relation $\langle a, \sigma \rangle \rightarrow_{\text{arith}} \langle a', \sigma \rangle$ performs exactly one arithmetic-expression step. Numerals are values and have no expression step. In the operator rules below, $\circ \in \{+, -, *, /\}$; an [A-Op] step exists only when $\text{arith}_m(\circ, n_1, n_2)$ is defined.

$$\begin{array}{c}
 \frac{}{\langle x, \sigma \rangle \rightarrow_{\text{arith}} \langle \sigma(x), \sigma \rangle} \text{[A-Var]} \\
 \\
 \frac{\langle a_1, \sigma \rangle \rightarrow_{\text{arith}} \langle a'_1, \sigma \rangle}{\langle a_1 \circ a_2, \sigma \rangle \rightarrow_{\text{arith}} \langle a'_1 \circ a_2, \sigma \rangle} \text{[A-Op-L]} \\
 \\
 \frac{\langle a_2, \sigma \rangle \rightarrow_{\text{arith}} \langle a'_2, \sigma \rangle}{\langle n_1 \circ a_2, \sigma \rangle \rightarrow_{\text{arith}} \langle n_1 \circ a'_2, \sigma \rangle} \text{[A-Op-R]} \\
 \\
 \frac{}{\langle n_1 \circ n_2, \sigma \rangle \rightarrow_{\text{arith}} \langle \text{arith}_m(\circ, n_1, n_2), \sigma \rangle} \text{[A-Op]} \\
 \\
 \frac{\langle i, \sigma \rangle \rightarrow_{\text{arith}} \langle i', \sigma \rangle}{\langle u[i], \sigma \rangle \rightarrow_{\text{arith}} \langle u[i'], \sigma \rangle} \text{[A-Arr-Idx]} \\
 \\
 \frac{0 \leq z < |\sigma(u)|}{\langle u[z], \sigma \rangle \rightarrow_{\text{arith}} \langle \sigma(u)(z), \sigma \rangle} \text{[A-Arr]}
 \end{array}$$

These contexts enforce left-to-right evaluation: [A-Op-R] applies only after the left operand is a numeral, and [A-Op] only after both operands are numerals.

Small-step Boolean expressions

The relation $\langle b, \sigma \rangle \rightarrow_{\text{bool}} \langle b', \sigma \rangle$ performs exactly one Boolean-expression step. Boolean literals are values. Below, $\odot \in \{=, \leq\}$ and $\diamond \in \{\text{and}, \text{or}\}$.

$$\begin{array}{c}
 \frac{\langle a_1, \sigma \rangle \rightarrow_{\text{arith}} \langle a'_1, \sigma \rangle}{\langle a_1 \odot a_2, \sigma \rangle \rightarrow_{\text{bool}} \langle a'_1 \odot a_2, \sigma \rangle} \text{[B-Cmp-L]} \\
 \\
 \frac{\langle a_2, \sigma \rangle \rightarrow_{\text{arith}} \langle a'_2, \sigma \rangle}{\langle n_1 \odot a_2, \sigma \rangle \rightarrow_{\text{bool}} \langle n_1 \odot a'_2, \sigma \rangle} \text{[B-Cmp-R]} \\
 \\
 \frac{}{\langle n_1 \odot n_2, \sigma \rangle \rightarrow_{\text{bool}} \langle \text{cmp}(\odot, n_1, n_2), \sigma \rangle} \text{[B-Cmp]} \\
 \\
 \frac{\langle b, \sigma \rangle \rightarrow_{\text{bool}} \langle b', \sigma \rangle}{\langle \text{not } b, \sigma \rangle \rightarrow_{\text{bool}} \langle \text{not } b', \sigma \rangle} \text{[B-Not-Step]} \\
 \\
 \frac{}{\langle \text{not true}, \sigma \rangle \rightarrow_{\text{bool}} \langle \text{false}, \sigma \rangle} \text{[B-Not-T]} \quad \frac{}{\langle \text{not false}, \sigma \rangle \rightarrow_{\text{bool}} \langle \text{true}, \sigma \rangle} \text{[B-Not-F]} \\
 \\
 \frac{\langle b_1, \sigma \rangle \rightarrow_{\text{bool}} \langle b'_1, \sigma \rangle}{\langle b_1 \diamond b_2, \sigma \rangle \rightarrow_{\text{bool}} \langle b'_1 \diamond b_2, \sigma \rangle} \text{[B-Con-L]} \\
 \\
 \frac{\langle b_2, \sigma \rangle \rightarrow_{\text{bool}} \langle b'_2, \sigma \rangle}{\langle t_1 \diamond b_2, \sigma \rangle \rightarrow_{\text{bool}} \langle t_1 \diamond b'_2, \sigma \rangle} \text{[B-Con-R]} \\
 \\
 \frac{}{\langle t_1 \diamond t_2, \sigma \rangle \rightarrow_{\text{bool}} \langle \text{con}(\diamond, t_1, t_2), \sigma \rangle} \text{[B-Con]}
 \end{array}$$

Rule [B-Con-R] deliberately has no short-circuit side condition. Thus both operands of and and or are evaluated.

Small-step commands

The relation $\langle c, \sigma \rangle \rightarrow_{\text{cmd}} \langle c', \sigma' \rangle$ performs exactly one command step. Unlike expression steps, a command step may change the state.

$$\begin{array}{c}
\frac{\langle a, \sigma \rangle \rightarrow_{\text{arith}} \langle a', \sigma \rangle}{\langle x := a, \sigma \rangle \rightarrow_{\text{cmd}} \langle x := a', \sigma \rangle} \text{[S-Asgn-Step]} \\
\\
\frac{}{\langle x := n, \sigma \rangle \rightarrow_{\text{cmd}} \langle \text{skip}, \sigma[x \mapsto n] \rangle} \text{[S-Asgn]} \\
\\
\frac{\langle c_1, \sigma \rangle \rightarrow_{\text{cmd}} \langle c'_1, \sigma' \rangle}{\langle c_1; c_2, \sigma \rangle \rightarrow_{\text{cmd}} \langle c'_1; c_2, \sigma' \rangle} \text{[S-Seq]} \\
\\
\frac{}{\langle \text{skip}; c_2, \sigma \rangle \rightarrow_{\text{cmd}} \langle c_2, \sigma \rangle} \text{[S-Seq-Done]} \\
\\
\frac{\langle b, \sigma \rangle \rightarrow_{\text{bool}} \langle b', \sigma \rangle}{\langle \text{if } b \text{ then } c_1 \text{ else } c_2, \sigma \rangle \rightarrow_{\text{cmd}} \langle \text{if } b' \text{ then } c_1 \text{ else } c_2, \sigma \rangle} \text{[S-If-Step]} \\
\\
\frac{}{\langle \text{if true then } c_1 \text{ else } c_2, \sigma \rangle \rightarrow_{\text{cmd}} \langle c_1, \sigma \rangle} \text{[S-If-True]} \\
\\
\frac{}{\langle \text{if false then } c_1 \text{ else } c_2, \sigma \rangle \rightarrow_{\text{cmd}} \langle c_2, \sigma \rangle} \text{[S-If-False]} \\
\\
\frac{}{\langle \text{while } b \text{ do } c, \sigma \rangle \rightarrow_{\text{cmd}} \langle \text{if } b \text{ then } (c; \text{while } b \text{ do } c) \text{ else skip}, \sigma \rangle} \text{[S-While]} \\
\\
\frac{\langle i, \sigma \rangle \rightarrow_{\text{arith}} \langle i', \sigma \rangle}{\langle u[i] := a, \sigma \rangle \rightarrow_{\text{cmd}} \langle u[i'], \sigma \rangle} \text{[S-Arr-Idx]} \\
\\
\frac{0 \leq z < |\sigma(u)| \quad \langle a, \sigma \rangle \rightarrow_{\text{arith}} \langle a', \sigma \rangle}{\langle u[z] := a, \sigma \rangle \rightarrow_{\text{cmd}} \langle u[z] := a', \sigma \rangle} \text{[S-Arr-Step]} \\
\\
\frac{0 \leq z < |\sigma(u)|}{\langle u[z] := n, \sigma \rangle \rightarrow_{\text{cmd}} \langle \text{skip}, \sigma[u[z] \mapsto n] \rangle} \text{[S-Arr]}
\end{array}$$

There is no command-step rule for skip: it is already final. For an array write, the index must reduce to a numeral and pass the bounds check before [S-Arr-Step] may reduce the value expression.

The optional bonus adds exactly these two rules; it does not change the deterministic relation used in Parts B and C:

$$\frac{}{\langle c_1 [] c_2, \sigma \rangle \rightarrow_{\text{cmd}} \langle c_1, \sigma \rangle} \text{[S-Choice-L]} \quad \frac{}{\langle c_1 [] c_2, \sigma \rangle \rightarrow_{\text{cmd}} \langle c_2, \sigma \rangle} \text{[S-Choice-R]}$$

There is no big-step rule for choice in the required language: a choice can have more than one final state. The bonus therefore uses the separate bounded exploration interface described later.

1 Part A — Big-step semantics

[25 marks]

The definitive rules above use the big-step command judgement

$$\langle c, \sigma \rangle \Downarrow \sigma'.$$

It relates a command and an initial state directly to a final state. In this part, you are asked to implement that relation and to construct the derivation that justifies every result.

The three functions are called `big_a`, `big_b`, and `big_c` in Python, and `bigA`, `bigB`, and `bigC` in Java and C++. Their declarations make the three judgements explicit:

Function	Typed inputs	Result
<code>big_a</code>	<code>AExp</code> , <code>State</code> , arithmetic mode	integer <code>EvalResult</code>
<code>big_b</code>	<code>BExp</code> , <code>State</code> , arithmetic mode	Boolean <code>EvalResult</code>
<code>big_c</code>	<code>Command</code> , <code>State</code> , arithmetic mode	state <code>EvalResult</code>

An `EvalResult[T]` contains two named fields: `value`, of type `T`, and `derivation`, of type `Derivation`. Thus `big_a` does not return an untyped tuple whose components you must remember. Its result explicitly pairs the integer denoted by the expression with the proof tree that establishes that value. We suggest the following order.

In `int` mode, the integer type is arbitrary precision: Python uses its built-in `int`, Java uses `BigInteger`, and C++ uses the supplied `imp::Integer` alias for the class in `integer.hpp`. Thus the implementation matches the mathematical integers in the rules; it does not silently overflow at 64 bits. In `int32` mode, the supplied arithmetic helper instead wraps each operation result to the signed 32-bit range.

Step A1: arithmetic expressions and assignment

First implement the core cases of arithmetic evaluation: `[Num]`, `[Var]`, `[Add]`, `[Sub]`, and `[Mul]`. Then implement the assignment case `[Asgn]` in the command evaluator. Assignment is needed because the first tests ask your program to execute a command; tests such as `a01` use an assignment to expose the value produced by an arithmetic expression.

Begin with `a01`, then proceed through `a04`:

```
python3 validate.py --cmd "python3 submission/solution.py" --only a01 tests/public
python3 validate.py --cmd "python3 submission/solution.py" --only a02 tests/public
python3 validate.py --cmd "python3 submission/solution.py" --only a03 tests/public
python3 validate.py --cmd "python3 submission/solution.py" --only a04 tests/public
```

How to implement `big_a`

Open the file for your language and find the named function:

- Python: `submission/big_step.py`, function `big_a`;
- Java: `submission/BigStep.java`, method `bigA`; and
- C++: `submission/big_step.cpp`, function `bigA`.

The function receives an expression that has already been decoded into one of four AST classes: `Num`, `Var`, `BinaryAExp`, or `ArrayRead`. Part A initially needs only the first three; array reads are added in Part D.

Begin with the `Num` case. A numeral needs no recursive premise: its value is stored in the AST node itself. In Python, the complete first case is:

```
if isinstance(expression, Num):
    value = expression.value
    derivation = Derivation.expression("Num", state, value)
    return EvalResult(value, derivation)
```

The test `isinstance(expression, Num)` asks whether the typed AST node is a numeral. `expression.value` is the integer written in that node. For the `Num(7)` node in test `a01`, `Derivation.expression` constructs this complete one-node derivation:

$$\frac{}{\langle 7, \sigma \rangle \Downarrow 7} [\text{Num}].$$

The empty space above the line means that `[Num]` has no premises. The input state σ is the object named `state` in the function, and the numeral on the right is the object field named `value`. Finally, `EvalResult` returns that value and this derivation together.

The same case begins with expression `instanceof Model.Num` in Java. In C++, use `std::get_if<imp::Num>(&expression->node)` to ask whether the variant currently contains an `imp::Num`. The comments immediately above `bigA` give the corresponding field names and result constructor.

Next implement `Var`. The AST field gives the variable's name. Read it with the state helper for your language:

Python	<code>state.read_variable(name)</code>
Java	<code>state.readVariable(name)</code>
C++	<code>state.readVariable(name)</code>

These helpers implement the total-map convention: an unmentioned variable reads as 0. Construct a leaf derivation labelled `[Var]` and return it with the value.

For `BinaryAExp`, the rule has two premises. Your implementation should perform these actions in order:

1. call `big_a` recursively on left in the original state;
2. call `big_a` recursively on right in the same state;
3. apply the node's operator to the two returned values using the supplied arithmetic helper;
4. select the rule name `Add`, `Sub`, `Mul`, or `Div`;
5. construct a derivation whose premises are the left derivation followed by the right derivation; and
6. return the computed value and this new derivation.

Expressions do not update the state. Both recursive calls therefore receive the same state; only the value and derivation fields of their results are used. This is the direct programmatic reading of the big-step rule

$$\frac{\langle a_1, \sigma \rangle \Downarrow n_1 \quad \langle a_2, \sigma \rangle \Downarrow n_2}{\langle a_1 + a_2, \sigma \rangle \Downarrow n_1 + n_2} [\text{Add}].$$

Once these cases work, implement assignment in `big_c/bigC`: evaluate the assignment's expression with `big_a`, update the state with its supplied typed helper, and make an `[Asgn]` derivation whose single premise is the expression derivation. This is why `a01` tests `big_a` through a command rather than sending an arithmetic expression as a top-level request.

The state helpers supplied with each adapter already implement the convention stated in the definitive rules: states are total, so an unmentioned variable reads as 0. They also keep states in *canonical form* by omitting variables whose value is 0. Use these helpers rather than accessing or updating the variable map differently in each rule.

Step A2: boolean expressions and conditionals

Next implement [True], [False], [Eq], [Leq], [Not], [And], and [Or]. Once boolean evaluation works, implement [If-True] and [If-False].

The connectives and and or in IMP are *strict*: evaluate both operands and then apply the connective. Do not use the short-circuit operators of your implementation language to decide whether the second recursive call is made.

Tests a05, a06, a09, and a15 are useful while working on these cases.

Step A3: the remaining commands

Finally implement [Skip], [Seq], [While-False], and [While-True]. The [While-True] derivation has three premises, in this order: evaluation of the guard, execution of the body, and execution of the remaining loop.

A derivation is also represented as a recursive JSON tree. Each node records the rule used, the input state, and a list of premise nodes. An expression node has a val field for the value produced; a command node has an out field for the resulting state. For example:

```
{ "rule": "Asgn", "in": { "vars": { "y": 5 }, "arrays": {} },
  "out": { "vars": { "x": 7, "y": 5 }, "arrays": {} },
  "prem": [ ... ] }
```

Here rule names the rule [Asgn]; in and out are the states before and after the assignment; and prem is the list of derivations for the rule's premises. A state stores scalar variables in vars and arrays in arrays. The ellipsis abbreviates the premise derivation for the expression 7; your actual JSON response must include that node.

You may add a subj string to make rendered proof trees easier to read, but this field is optional and ungraded.

After completing Part A, the following command should report graded: 15/15 passed:

```
python3 validate.py --cmd "python3 submission/solution.py" --part A tests/public
```

The following command asks render.py to read the expected response in a11 and display its loop derivation. It does not run your implementation:

```
python3 render.py --from-test tests/public/a11.json
```

Big-step evaluation and nontermination What happens if your recursive evaluator is called on while true do skip? The rules do not produce a finite derivation for this program. Written Question 2 asks you to compare this case with a stuck program.

2 Part B — Small-step semantics

[25 marks]

The definitive rules define three small-step relations, for arithmetic expressions, Boolean expressions, and commands. A small step performs one piece of work. In this part, you are asked first to implement one step and then to write an execution function that repeatedly takes steps.

If Part A is complete and Part B is untouched, –part B should still pass only b01. Work through the following stages.

Step B1: one arithmetic or command step

Implement `step_a` and the assignment cases of `step_c` (`stepA` and `stepC` in Java/C++). Use tests `b02`, `b03`, and `b04` in this order.

For example, `b02` begins with

$$\langle x := y + 2, [y \mapsto 5] \rangle$$

and expects exactly one step:

$$\langle x := 5 + 2, [y \mapsto 5] \rangle.$$

The addition is not performed in the same step. The derivation of this one transition uses `[S-Asgn-Step]`, `[A-Op-L]`, and `[A-Var]`.

In Python, the result of `step_a` is still an arithmetic-expression object, not a bare integer. For example, `Num(value)` constructs a new numeral node (equivalently, `Num(value=value)`). Likewise, `step_c` returns a `Configuration` object. These are equivalent positional and keyword constructor calls:

```
Configuration(next_command, next_state)

Configuration(
    command=next_command,
    state=next_state,
)
```

Both construct the pair $\langle \text{next_command}, \text{next_state} \rangle$. If a recursive command step returns a configuration named `inner`, its two fields are `inner.command` and `inner.state`; the configuration itself is not a command.

Hint. The left operand of an operator is reduced until it is a numeral; only then may the right operand take a step. Apply the operator only when both operands are numerals. This is the left-to-right order fixed by `[A-Op-L]`, `[A-Op-R]`, and `[A-Op]`.

Each premise calls the helper for that premise’s syntactic category: arithmetic premises call `step_a`, Boolean premises call `step_b`, and command premises call `step_c`. Pass the expression or command, the state, and the arithmetic mode on every recursive call. The returned residual is then placed back into a newly constructed outer AST node; one small step never recursively evaluates the entire tree.

Step B2: boolean expressions and the remaining commands

Implement `step_b`, followed by sequencing, conditionals, and while loops in `step_c`. Follow the rule premises literally: an outer rule takes one premise step and rebuilds the surrounding expression or command.

Keep the following cases separate.

- An outermost skip is final. The supplied solution entry point checks this before calling `step_c`, so `step_c` normally receives only non-final commands. Final and stuck configurations both lack a successor, but they are distinguished by checking for skip first.
- `[S-Seq-Done]` is one step: skip; `c` steps to `c`. The outer command here is a sequence, not skip, so it is not final.

- A while loop takes one [S-While] step to its unfolded conditional. Do not evaluate the guard during this step.
- and and or remain strict in the small-step semantics. The right operand is reduced after the left operand becomes a boolean value.

The optional return values have category-specific meanings. In `step_a`, no residual means that the input is already a `Num`; in `step_b`, it means that the input is already a `Bool`. For `step_c`, finality has already been checked by the caller, so no successor means that the non-final command is stuck. Do not use a missing successor to classify skip.

Tests `b05`–`b14` separate these cases. In particular, `b09` and `b10` distinguish strict evaluation from short-circuiting.

Step B3: run a complete execution

Implement `run`. It repeatedly calls the command-step function and records every configuration, starting with the initial one. It reports one of three outcomes:

Status	When to return it
terminated	the current command is skip
stuck	the command is not final and has no successor
budget	the permitted number of steps has been taken

Test the terminal condition *before* testing the budget. Thus `x := 1` terminates with budget 1: after its one permitted step, the current command is skip. A budget result says only that the execution continued for that many steps; it does not prove divergence.

The loop order is therefore:

1. inspect the current configuration and return `terminated` if its command is skip;
2. return `budget` if the permitted number of steps has been used;
3. otherwise try one call to `step_c`;
4. return `stuck` if no successor exists, or append the successor and increment the step count.

The configuration list begins with the input configuration, so after n successful transitions it contains $n + 1$ entries.

Stuckness can also be detected inside a supplied helper. Division by zero in the arithmetic helper, and an out-of-bounds array read or write in the state helper, raise the supplied `Stuck` exception. The execution function must turn that exception into a `stuck RunResult`; a non-final `step_c` call that returns no successor has the same status with the generic reason “no rule applies.” For example, if $y = 2$, execution of `x := y / 0` first reduces y to 2, then becomes stuck when the division is attempted.

Tests `b11`–`b14` use `run` mode so that they can observe an expression over several small steps. Consequently, an unimplemented `run` masks those expression tests with a `TODO` Part B: `run` failure even when earlier one-step tests pass.

Use `c01` first. Its execution should have four configurations and three steps. You can render the expected execution as a table:

```
python3 render.py --from-test tests/public/c01.json
```

After completing Part B, this command should report graded: 21/21 passed:

```
python3 validate.py --cmd "python3 submission/solution.py" --part B tests/public
```

3 Part C — Detecting repeated configurations

[10 marks]

Part B stops when a budget is exhausted, but the resulting budget status does not tell us whether the program will eventually terminate. In this part, you are asked to detect one particular proof of divergence: the same configuration has occurred twice.

Extend the execution loop from Part B by implementing `classify`. It reports:

Status	What it means
terminated	reached $\langle \text{skip}, \sigma' \rangle$
stuck	not final, and no rule applies
diverges	a configuration has repeated
unknown	the budget ended without a repeat

Why does a repeat prove divergence here? The required small-step relation is deterministic. Once the unique execution returns to an earlier configuration, it must take the same sequence of steps again.

We suggest the following implementation. Maintain a map from each configuration to the step at which it was first seen. At each iteration:

1. return `terminated` if the current command is final;
2. return `diverges` if the current configuration is already in the map;
3. return `unknown` if the budget has been exhausted;
4. otherwise record the configuration and try to take one step; return `stuck` if no successor exists or if the supplied helper raises `Stuck`.

As in `run`, a failed step attempt does not increment the step count.

Compare configurations by *value*, not by object identity. The supplied state helpers are relevant again: if one state retains $x:0$ and another omits it, two representations of the same total state may prevent you from detecting a cycle.

Start with `d01`, the loop `while true do skip`. It returns to its initial configuration after three steps. Then compare it with `d02` and `d03`, which continue without repeating a concrete configuration.

After completing Part C, the following should report graded: 6/6 passed:

```
python3 validate.py --cmd "python3 submission/solution.py" --part C tests/public
```

4 Part D — Division, wrapping arithmetic, and arrays

[20 marks]

Parts A–C use the core language. In this part, you will extend both semantics with division, signed 32-bit arithmetic, and arrays. Complete Parts A–C first: Part D reuses both evaluators and the complete-execution functions, and test `e14` also calls the classifier from Part C.

Step D1: division

Add division to both arithmetic evaluators: the big-step evaluator and the small-step relation. Division truncates *toward zero*, as in C: $7/2 = 3$ and $-7/2 = -3$. Division by zero is stuck.

Each adapter supplies a division helper that implements these details. Use it instead of the host language's operator: Python's `//`, for example, rounds negative results differently.

Begin with e01 and e02. Tests e12 and e13 check that division by zero becomes stuck. Because `and` and `or` are strict, $x = 0$ or $1/x = 0$ is stuck when x is 0: evaluating the left operand to `true` does not skip the division on the right. In the core language, expressions are total, so eager and short-circuit connectives produce the same truth values; division makes their operational difference observable.

Step D2: wrapping arithmetic

Every request has an `arith` field. The value `int` selects the mathematical integers $\mathbb{Z}$. The value `int32` selects signed 32-bit arithmetic. Under `int32`, the *result* of `+`, `-`, `*`, and `/` is wrapped into $[-2^{31}, 2^{31} - 1]$. A numeral itself is not wrapped.

The adapters already supply a wrapping helper. Arrange your implementation so that the semantic rule chooses an arithmetic policy; do not duplicate the inference rules for `int` and `int32`. Remember to use the same policy in both big-step and small-step evaluation.

Tests e03 and e04 compare

$$m := (lo + hi) / 2 \quad \text{and} \quad m := lo + (hi - lo) / 2$$

from $[lo \mapsto 1, hi \mapsto 2^{31}-1]$. Over $\mathbb{Z}$ the two expressions agree. Under `int32`, the first addition wraps and the two programs need not agree. Tests e16–e18 also check wrapping in the small-step relation.

Step D3: arrays

The state now has a second component containing arrays. The initial state fixes the available array names, their lengths, and their contents. Arrays are zero-indexed and never resize. An out-of-bounds read or write is stuck.

Add `[Arr-Read]` to big-step arithmetic evaluation and `[Arr-Write]` to big-step command evaluation. For small-step evaluation, implement `[A-Arr-Idx]`, `[A-Arr]`, `[S-Arr-Idx]`, `[S-Arr-Step]`, and `[S-Arr]`. All seven rules, including their bounds side conditions, appear in the definitive semantics section.

These rules also specify an evaluation order for `a[i] := e`: first evaluate the index, then check that it is in bounds, and only then evaluate the value. Thus, on an array of length three, `a[5] := 1/0` becomes stuck on the index; the division is never evaluated. Use the same order in your big-step implementation.

Use e06–e11 for the basic array cases and e14, e15 for stuckness and a complete small-step array read.

After completing Part D, the following should report graded: 18/18 passed:

```
python3 validate.py --cmd "python3 submission/solution.py" --part D tests/public
```

At this point all 60 required implementation tests included in the archive should pass. If you ask `validate.py` to run every file in `tests/public/`, it will report graded: 60/64 passed: the directory contains these 60 required tests and four bonus tests that you have not yet implemented.

5 Written questions

[20 marks]

Submit Section 5 as answers.pdf, a legible scan of your handwritten work. Show the intermediate reasoning in every proof, including the induction hypothesis and semantic rules used. Typed submissions are permitted for approved accommodations. You may use AI tools subject to the disclosure policy, but you remain responsible for the argument and may be asked to explain it briefly.

Question 1 [6 marks] Let $D = \text{while } 1 \leq x \text{ do } x := x - 1$, with arithmetic over the mathematical integers. Prove:

For every state σ with $\sigma(x) \geq 0$, there is a state σ' such that $\langle D, \sigma \rangle \Downarrow \sigma'$ and $\sigma' = \sigma[x \mapsto 0]$.

The equation $\sigma' = \sigma[x \mapsto 0]$ says all three things at once: x is 0, every other variable is unchanged, and every array is unchanged.

Use induction on $N = \sigma(x)$. State an induction hypothesis quantified over *every* state in which x has value N , and indicate which big-step rules each case uses.

Question 2 [6 marks] Let σ contain an array a of length 3, and consider

$$S = x := a[3] \quad \text{and} \quad L = \text{while true do skip}.$$

For each of S and L :

- (a) determine whether there is a state σ' with $\langle S, \sigma \rangle \Downarrow \sigma'$ (respectively, $\langle L, \sigma \rangle \Downarrow \sigma'$);
- (b) classify its small-step execution from σ as terminating, stuck, or diverging; and
- (c) justify both answers directly from the rules.

Then explain why the *absence of a big-step result from σ* does not distinguish stuckness from divergence.

Question 3 [8 marks] Consider

$$N = \text{while true do } (i := i + 1; j := j - 1).$$

- (a) Let $\gamma_0 \rightarrow \gamma_1 \rightarrow \dots \rightarrow \gamma_n$ be a finite execution prefix satisfying $\gamma_p = \gamma_q$ for some $0 \leq p < q \leq n$. Prove that the cycle from γ_p back to itself can be repeated to construct an infinite execution. Then explain why determinism implies that the unique maximal execution from γ_0 follows this cycle. State exactly where determinism is used.
- (b) Starting from a state with $\sigma(i) = v$ and $\sigma(j) = w$, prove by induction on k that after $10k$ steps the execution is back at the head of the loop, in the state $\sigma[i \mapsto v + k, j \mapsto w - k]$.
- (c) Prove that no configuration in this execution ever repeats. Conclude that N diverges, and that the detector of Part C returns unknown for every finite budget.

Parts (a) and (c) ask you to establish the two boundaries of the detector: when it reports diverges, that conclusion is sound; nevertheless, some diverging executions contain no repeated configuration. For part (c), a useful route is to ask what one repeated configuration would make the rest of this deterministic execution look like, and then compare that consequence with part (b).

6 Bonus — Non-deterministic choice

[up to 10 marks]

For the bonus, extend the language with $c_1 [] c_2$. It may take an [S-Choice-L] or [S-Choice-R] step, as shown in the definitive semantics section. Exercise 2.12 is a useful direct precursor to the bounded reachability reasoning used here, but the statement below is self-contained.

The step relation is now non-deterministic. Keep the deterministic step function from Part B unchanged. Instead:

1. implement a separate `step_all` that returns every successor configuration (Java already declares `stepAll`; in C++, add a helper of this form if useful);
2. implement `explore` as a bounded breadth-first search;
3. collect final states in canonical form and remove duplicates; and
4. report whether a stuck configuration was found and whether the search was truncated by its budget.

Start with `f01`, which has two branches leading to two different final states. Tests `f02` and `f03` check deduplication. After completing the bonus, `-part bonus` should report graded: 4/4 passed. Running `validate.py` on the entire `tests/public/` directory should then report graded: 64/64 passed.

Do not extend Part C's cycle detector to the non-deterministic fragment. "Some execution diverges" and "every execution diverges" are different properties, and this homework does not ask you to decide either one.

Bounded reachability With both loops and choice, a bounded search need not find every reachable final state. The `explore` response therefore contains the final states found within the budget and a truncated flag. Every bonus test is loop-free and supplies a budget large enough to finish the search.

Malformed requests

A request is malformed if, for example, it names an array absent from the initial state, or an `int32` numeral or initial value lies outside the representable range. Malformed input is different from a well-formed program that becomes stuck during execution.

The four files in `tests/malformed/` document inputs that the validation system rejects before starting your program. They are ungraded. You do not need to implement malformed-input checking.

Use of AI coding assistants

You may use AI coding assistants for this homework. You remain responsible for the submitted implementation and must be able to explain and modify it. Fill in the supplied `AI-USE.md` template, naming the tools you used, describing substantive assistance, and explaining one suggestion you accepted and one you rejected (or saying that you rejected none). Submit the file even if you used no such tool; in that case, say so in one line. Disclosure does not affect your mark.

What to submit

Submit one archive containing:

1. the submission/ directory containing the working copy of your chosen adapter and all of its source files;
2. a file named RUN, whose single line is the command used to start your program, exactly as it appears after `validate.py -cmd`;
3. a Makefile if your submission must be compiled. Its default target should build the executable or class files needed by the command in RUN;
4. answers.pdf, a legible scan of your handwritten Section 5 work unless you have an approved accommodation; and
5. the completed AI-USE.md.

The marker uses the line in RUN in the same way that you used the value of `-cmd` above. For example, a Python submission may contain `python3 submission/solution.py`; a Java submission may contain `java -cp build/java Solution`.

The top level of your archive should therefore contain `submission/`, `RUN`, `answers.pdf`, and `AI-USE.md`, together with a Makefile when compilation is required. Do not replace `submission/` with one source file: the model, codec, arithmetic, and semantic files are all needed to run your program.

For example, the archive should unpack to this structure:

<code>submission/</code>	all files from your chosen adapter, after your changes
<code>RUN</code>	one command that starts your program
<code>Makefile</code>	required for Java and C++; not needed for Python
<code>answers.pdf</code>	a legible scan of your handwritten Section 5 answers
<code>AI-USE.md</code>	your completed disclosure

The contents of RUN are one of the following, depending on your language:

```
# Python
python3 submission/solution.py

# Java
java -cp build/java Solution

# C++
./build/solution
```

For Java or C++, the default target of Makefile must compile the source files under `submission/` and produce the class files or executable used by the corresponding line above. The marker first runs `make` and then starts the command in RUN. Python submissions do not need a Makefile.

Before submitting, make a fresh copy of the released `cs4211-hw1` directory and unpack your submission archive into that copy. Build it if necessary, then run `validate.py` against `tests/public` using the line from your RUN file. This final check often catches a missing source file, helper, or build command. If the submitted program cannot be run, only the written questions can receive marks.